

TP – Échantillonnage et analyse spectrale

Travail préparatoire

Lire l'énoncé du TP et ses annexes et effectuer l'application 1, l'application 3

Appel prof — Évaluation

$\frac{1}{8}$

Le cahier de TP sera rendu à la fin de la séance et le compte-rendu évalué.

les manipulations faites sont précisément décrites

/ 3

les observations et résultats bruts sont consignés (papier, informatique, ..)

/ 2

les traitements, calculs, ... sont explicités

/ 2

une conclusion est apportée

/ 3

Total

/ 10

1. Introduction

L'objectif de ce TP est d'étudier le filtre de Wien et sa fonction de transfert.

2. Réalisation et étude préliminaire

Le filtre de Wien sera réalisé avec les composants accessibles dans la salle de TP de sorte que

- sa fréquence caractéristique soit comprise entre 10 kHz et 20 kHz ;
- sa résistance soit comprise entre 2 et 20 .

Application 1



Choisir des valeurs pour R et C .

Les composants utilisés en TP ont une incertitude-type de 5%. Calculer l'incertitude sur la fréquence caractéristique f_0 .

Évaluation

$\frac{1}{8}$

Choix des valeurs de R et C et calcul de l'incertitude sur la fréquence caractéristique f_0 .

les valeurs de

/ 2

les signaux prennent toute la place

/ 2

on voit quelques périodes

/ 2

critère sur l'amplitude

/ 2

estimation de la fréquence

/ 2

Total

/ 10

Manipulation 1



Réaliser le filtre de Wien sur une platine d'essai.

Relier son entrée à un GBF et observer sa sortie sur un oscilloscope.

Vérifier rapidement et qualitativement que la nature du filtre et la fréquence caractéristique sont conformes aux attentes.

Appel prof — Évaluation

$\frac{1}{8}$

L'enseignant dérègle l'oscilloscope et le binôme le re-règle sans utiliser le bouton auto-scale.

les signaux ne défile pas

/ 2

les signaux prennent toute la place

/ 2

on voit quelques périodes

/ 2

critère sur l'amplitude

/ 2

estimation de la fréquence

/ 2

Total

/ 10

Appel prof — Évaluation		$\frac{1}{8}$
L'enseignant dérègle l'oscilloscope et le binôme le re-règle sans utiliser le bouton auto-scale.	<i>les signaux ne défile pas</i>	/ 2
	<i>les signaux prennent toute la place</i>	/ 2
	<i>on voit quelques périodes</i>	/ 2
Le binôme montre ensuite comment il estime la fréquence caractéristique en utilisant la phase.	<i>critère sur la phase</i>	/ 2
	<i>estimation de la fréquence</i>	/ 2
Total		/ 10

3. Tracé et analyse du diagramme de Bode

Le diagramme de Bode (en gain et en phase) sera tracé après un relevé point par point réalisé à l'aide du GBF et de l'oscilloscope. Les tracés seront réalisés avec Python grâce à la bibliothèque pyplot, notamment la fonction `semilogx` de la bibliothèque `matplotlib.pyplot` qui fait le tracé avec les abscisses en échelle logarithmique. La fonction `semilogx` prend les mêmes arguments que la fonction `plot` à la place de laquelle elle s'utilise.

Application 2	
Expliquer pourquoi il est nécessaire de mesurer également l'amplitude de la tension d'entrée, issue du GBF, plutôt que de lire sa valeur sur l'écran de celui-ci.	

Manipulation 2	
Mesurer pour chaque point • l'amplitude de la tension d'entrée ; • l'amplitude de la tension de sortie ; • la fréquence des signaux ; • le retard de la sortie par rapport à l'entrée. On répartira les points pour des fréquences entre 10 Hz et 100 kHz en choisissant 5 points avant la fréquence caractéristique et 5 points après. Les données seront rentrées directement dans des variables dans Python. Tracer le diagramme de Bode en gain et en phase grâce à Python.	

Appel prof — Évaluation		$\frac{1}{8}$
L'enseignant dérègle l'oscilloscope et le binôme le re-règle sans utiliser le bouton auto-scale.	<i>les signaux ne défile pas</i>	/ 2
	<i>les signaux prennent toute la place</i>	/ 2
	<i>on voit quelques périodes</i>	/ 2
Le binôme montre ensuite comment mesure les grandeurs d'intérêt.	<i>mesure de l'amplitude</i>	/ 2
	<i>mesure du retard</i>	/ 2
Total		/ 10

Appel prof — Évaluation		$\frac{1}{8}$
L'enseignant regarde les courbes.	<i>courbe du gain en dB</i>	/ 2
	<i>courbe de la phase</i>	/ 2
	<i>titre</i>	/ 2
	<i>axes</i>	/ 2
	<i>points non reliés</i>	/ 2
Total		/ 10

L'analyse manuelle du diagramme de Bode ne peut être faite qu'à partir des asymptotes. Dans ce TP, comme les données sont numérisées, on peut faire des traitements plus avancés et notamment une régression sur les données expérimentales.

Pour cela, il est nécessaire de définir en Python la fonction de Bode théorique du filtre de Wien.

Application 3



Compléter le code suivant qui renvoie le gain en décibels du filtre de Wien.

```
1 def GdB(f, A, f0, Q):
2     # f : fréquence en Hz
3     # A : constante sans dimension
4     # f0 : fréquence caractéristique
5     # Q : facteur de qualité
6     return ...
```

Pour vérifier l'adéquation entre les diagrammes de Bode théorique et expérimental, et pour déterminer les paramètres A , f_0 et Q , on peut effectuer un ajustement (*fit* en anglais), c'est-à-dire chercher la courbe théorique passant au plus près des points expérimentaux.

Pour cela, on utilisera la fonction `curve_fit` de la bibliothèque `scipy.optimize`. Celle-ci prend en arguments

- la fonction à ajuster (son premier argument doit être l'abscisse et les suivants les paramètres à ajuster),
- la liste des abscisses des points expérimentaux,
- la liste des ordonnées des points expérimentaux,
- un paramètre optionnel `bounds` (tuple ou liste de tuples) qui correspond aux bornes inférieure et supérieure des paramètres à ajuster (par exemple, avec `bounds=(0,np.inf)`, les paramètres seront contraints à être positifs).

Elle renvoie un tuple dont le premier élément est la liste des valeurs optimales des paramètres de la fonction.

Manipulation 3



Écrire des instructions Python permettant d'effectuer l'ajustement de la fonction définie à l'application 3.

Les instructions devront afficher les valeurs optimales des paramètres A , f_0 et Q .

Pour savoir si l'ajustement est correct, il est d'usage de tracer sur le même graphique les points expérimentaux et la courbe théorique obtenue avec les paramètres optimaux.

Manipulation 4



Écrire des instructions Python permettant de tracer sur le même graphique les points expérimentaux et la courbe théorique obtenue avec les paramètres optimaux.

4. Prise en compte des incertitudes

Conclure sur la compatibilité ou non entre les valeurs attendues et les valeurs expérimentales, une estimation des incertitudes est indispensable. Reprendre une mesure faite sur l'oscilloscope.

Manipulation 5



Estimer l'incertitude relative sur la mesure du retard sur l'oscilloscope. On généralisera cette valeur à tous les points de mesure.

Estimer l'incertitude absolue sur la mesure de l'amplitude sur l'oscilloscope. On généralisera cette valeur à tous les points de mesure.

Évaluation

$\frac{1}{8}$

Présenter le calcul des incertitudes expérimentales sur l'amplitude.

<i>estimation présente</i>	/ 3
<i>estimation pertinente</i>	/ 4
<i>estimation précise</i>	/ 3
Total	/ 10

Évaluation		$\frac{1}{8}$
Présenter le calcul des incertitudes expérimentales sur le retard.	<i>estimation présente</i>	/ 3
	<i>estimation pertinente</i>	/ 4
	<i>estimation précise</i>	/ 3
	Total	/ 10

Il reste alors à propager cette incertitude sur la fréquence caractéristique f_0 , le facteur de qualité Q et sur A . On utilisera pour cela la méthode de Monte-Carlo.

Manipulation 6	
Implémenter la méthode de Monte-Carlo pour estimer l'incertitude sur la valeur de f_0 , A et Q .	

5. Conclusion

Il est maintenant possible de conclure sur l'adéquation des valeurs attendues aux valeurs mesurées pour f_0 , A et Q .

Application 4	
Les valeurs mesurées via l'ajustement des trois paramètres sont-elles cohérentes avec elles calculées à partir des valeurs des composants ? La conclusion sera rédigée après le calcul d'un écart normalisé.	

ANNEXE A : Fonctions Python utiles dans ce TP

<code>plt.clf</code> ¹	<code>plt.clf()</code> efface le graphique courant (pour tracer un nouveau graphique dans la même fenêtre)
<code>plt.plot</code> ¹	<code>plt.plot(x, y)</code> <code>plt.plot(x, y, format: str)</code> trace y en fonction de x ; format (ex. "r+", "g-") fixe couleur et style
<code>plt.semilogx</code> ¹	<code>plt.semilogx(x, y)</code> comme <code>plot</code> , avec l'axe des abscisses en échelle logarithmique
<code>plt.xlabel</code> ¹	<code>plt.xlabel(texte: str)</code> légende de l'axe des abscisses
<code>plt.ylabel</code> ¹	<code>plt.ylabel(texte: str)</code> légende de l'axe des ordonnées
<code>plt.title</code> ¹	<code>plt.title(texte: str)</code> titre du graphique
<code>plt.show</code> ¹	<code>plt.show()</code> affiche la fenêtre graphique, une fois tous les tracés effectués
<code>curve_fit</code> ²	<code>popt, pcov = curve_fit(f: fonction, xdata: array, ydata: array, p0: list) -> tuple</code> ajuste les paramètres de f (dont le premier argument est l'abscisse) aux données ; <code>popt</code> = paramètres optimaux, <code>p0</code> = valeurs initiales (facultatif)
<code>np.pi</code> ³	<code>np.pi</code> constante $\pi \approx 3.14159$
<code>np.linspace</code> ³	<code>x = np.linspace(début: float, fin: float, nombre_de_points: int) -> array</code> tableau de points régulièrement espacés entre <code>début</code> et <code>fin</code> (bornes incluses)
<code>np.logspace</code> ³	<code>f = np.logspace(début: float, fin: float, nombre_de_points: int) -> array</code> comme <code>linspace</code> , mais les points sont espacés logarithmiquement entre $10^{\text{début}}$ et 10^{fin}
<code>np.random.normal</code> ³	<code>x = np.random.normal(moyenne: float, écart_type: float) -> float</code> <code>x = np.random.normal(moyennes: array, écarts_types: array) -> array</code> valeurs aléatoires suivant une loi normale (gaussienne) distribuée autour de <code>moyenne</code> avec un <code>écart_type</code>

¹ `import matplotlib.pyplot as plt` ² `from scipy.optimize import curve_fit` ³ `import numpy as np`